内存受限下 Hash Join 算子 Spill-to-Disk 动态调度：个人补充版

版本：2026-10-28（转换与语义复核修订）。原题 PDF 标注更新于 2026-10-20 17:56。

本文件包含完整题干、参考框架、提交骨架与评分说明，并在相关章节加入【解释补充】。补充依据保存的参考实现和样例，不是官方修订。三个原文 C++ 代码块保持不变，仅规范化空白；错误或不精确的原代码注释在代码块外说明。表格重排、跨页样例空行清理和正文勘误均在本版说明，冻结原件不改。原始 PDF 是来源依据。

解释原则：未明确规定处按参考实现和样例处理；样例算术错误以逐步状态推演纠正。已有明确禁止规则仍遵守，不利用引擎漏检查。按此解释原则，合法动作下的核心状态转移已足以实现；文字与参考代码仍有差异，见第七节，不能据此认证隐藏评测实现完全相同。

来源：main 的 docs/statement-2026-10-20/statement.md（发布于 11b719b）；其 SHA-256 为 464a542ad56f0b81ebad23371846a705c202c4461589a5f37d65af5a65c99d69。原 PDF SHA-256 为 5438db9bcee81b9c5fde8cfbe46dbbf471d1d0e460e3c0eb75a637fefff6f41c。本次未重新在线核验官网。

出题组织：华为
接口专家：王碧华 wangbihua1@huawei.com
赛事背景
关联赛事：2026年度Hackathon软件难题挑战赛
内存受限下 Hash Join 算子 Spill-to-Disk 动态调度
来自技术领域：数据库 - 执行器

一、背景信息（赛题产生的业务背景）

在现代数据库执行引擎（如 DuckDB、Velox、Greenplum、OceanBase）中，Hash Join 和 Hash Aggregation 是最核心也是最消耗内存的算子。在处理大规模数据时，Hash 表的构建往往需要占用数 GB 甚至数十 GB 内存。当系统可用内存不足以容纳全部 Hash 数据时，执行引擎必须采用 Spill-to-Disk（落盘）策略：将部分 Hash 分区（Partition）从内存写出到磁盘，待后续 Probe 阶段需要时再从磁盘加载回来。
这一过程存在多重核心矛盾：
内存约束与数据规模的矛盾：系统总内存有限，而实际数据量可能远超内存容量，必须进行选择性落盘。
落盘时机与 I/O 开销的矛盾：过早落盘可能导致后续 Probe 阶段频繁触发磁盘读取，产生大量 I/O 开销；过晚落盘则可能导致内存溢出（OOM），系统直接崩溃。
分区选择与 I/O 抖动的矛盾：如果落盘的是即将被 Probe 访问的"热点"分区，会造成"刚写出又读回"的 I/O 抖动，严重浪费磁盘带宽。
多算子并发的资源竞争：当多个 SQL 查询并发执行时，不同算子之间争夺有限的内存资源，需要动态调配内存配额。
说明：本赛题中，K 个算子的操作按工作负载文件中的顺序逐一执行，不存在真正的并行执行。所谓"并发"指多个算子共享同一内存池，调度器需在算子间合理分配内存。
业界主流数据库均实现了 Spill-to-Disk 机制，但采用的策略各异：PostgreSQL 采用简单的批量落盘与逐分区重载，DuckDB 基于 Partition-level 粒度的自适应落盘与预取，Greenplum 则采用了多级 Grace Hash Join 策略。在数据倾斜、并发查询和有限内存的综合压力下，如何设计高效的 Spill 调度策略仍是数据库领域持续优化的方向。
本赛题要求选手在给定内存约束下，设计并实现 Hash Join 算子的 Spill-to-Disk 调度策略。系统模拟多个并发 Hash Join 算子的 Build 与 Probe 全流程，选手需实现调度器，决定何时落盘、落盘哪些分区、何时加载、如何在多算子间分配内存，在保证不发生 OOM 的前提下，最小化总执行延迟与磁盘 I/O 开销。

二、问题描述（现状、技术挑战等）

2.1 系统模型

系统包含以下核心参数：
总内存 M：系统可用于所有 Hash 分区的内存上限（单位：字节）。
磁盘带宽：
顺序写带宽 W（单位：字节/微秒）：向磁盘写入数据的速率。
顺序读带宽 R（单位：字节/微秒）：从磁盘读取数据的速率。
算子数量 K：系统中并发执行的 Hash Join 算子数量。
分区数 P：每个算子的 Hash 表被划分为 P 个分区（Partition 0 ~ P-1）。
Hash 函数：partition_id = key % P。
Miss 惩罚值：每次 Probe 未命中的额外惩罚（单位：微秒）。

2.2 数据结构

分区（Partition）属性：

- partition_id：分区编号，取值范围 [0, P-1]
- data_size：当前分区中数据的总大小（字节）
- in_memory：当前是否在内存中（true/false）

算子（Operator）属性：

- op_id：算子编号，取值范围 [0, K-1]
- phase：当前阶段，取值为 BUILD（构建阶段）或 PROBE（探测阶段）
- partitions：P 个分区的状态列表

系统状态：

- memory_used：当前所有 in_memory 分区的 data_size 之和
- memory_limit：等于 M

> 【解释补充：初始状态与 Hit】所有算子初始为 BUILD；每个分区初始 data_size=0、in_memory=true。空分区也可 Probe Hit。这里模拟分区驻留，不执行真实 Join 匹配，不要求 Probe key 曾被插入；PROBE 不改变分区大小，也不产生结果元组。

2.3 操作定义

工作负载文件包含三类操作，按序执行：
（1）INSERT op_id key size
向指定算子的 Hash 表插入一个元组。
执行逻辑：

1. 计算 partition_id = key % P。

2. 若该分区 in_memory = true：
若 memory_used + size ≤ M，直接插入，memory_used += size，分区 data_size += size。
若 memory_used + size > M，内存溢出——调度器必须先选择若干 in_memory 分区落盘，释放足够空间后再插入。

3. 若该分区 in_memory = false（已在磁盘上）：
该元组以增量追加方式写入磁盘（仅产生 size 字节的写 I/O），I/O 耗时 = size / W，分区 data_size += size，memory_used 不变。
后续若将该分区 LOAD 回内存，将加载包含历史落盘数据和新增数据的完整 data_size。
（2）SWITCH op_id
将指定算子从 BUILD 阶段切换到 PROBE 阶段。无时间开销，不计入周期性调度的操作计数。
（3）PROBE op_id key

在指定算子中探测一个键。
执行逻辑：

4. 计算 partition_id = key % P。

5. 若该分区 in_memory = true → 命中（Hit），耗时可忽略，不计入评分。

6. 若该分区 in_memory = false → 未命中（Miss）：
必须将该分区从磁盘加载到内存后才能执行探测。
若 memory_used + partition.data_size ≤ M，直接加载。
若 memory_used + partition.data_size > M，调度器必须先选择若干 in_memory 分区落盘，释放足够空间后再加载。
加载耗时 = partition.data_size / R，加载后 in_memory = true，memory_used += partition.data_size。
累计 miss_count += 1。

【解释补充：回调时机】overflow 看到插入前状态。磁盘分区发生 PROBE 时，无论空闲空间是否足够，都会调用 onPartitionMiss；进入回调前 miss_count 已加一，加载完成不再记 Hit。同一输入可先触发 overflow/miss，再触发 periodic；后者看到当前输入及其动作完成后的状态。SWITCH 不计入 B 已是原题明文约束。

【解释补充：周期与结束】B 的计数是全部算子共享的 INSERT/PROBE 总数，SWITCH 不重置计数。末尾操作若达到 B，仍触发 periodic，其动作照常计费。没有完成后自动回收算子内存的机制。

2.4 落盘与加载操作

落盘（SPILL）：将一个 in_memory 分区的数据写出到磁盘。
耗时 = partition.data_size / W。
落盘后：in_memory = false，memory_used -= partition.data_size。
分区的 data_size 不变（数据仍存在磁盘上）。
加载（LOAD）：将一个 on-disk 分区的数据从磁盘读入内存。
前提：memory_used + partition.data_size ≤ M（否则需先落盘其他分区）。
耗时 = partition.data_size / R。
加载后：in_memory = true，memory_used += partition.data_size。
约束：
任意时刻，memory_used ≤ M，否则判定为 OOM，该测试用例得 0 分。
分区具有原子性：一个分区的数据要么全部在内存中，要么全部在磁盘上。
一个算子处于 BUILD 阶段时，其分区只能被落盘；处于 PROBE 阶段时，其分区可以被落盘或加载。
禁止落盘当前正在插入的目标分区：当 onMemoryOverflow 因某个 INSERT 触发时，该 INSERT 所写入的目标分区（由 op_id 和 partition_id 指定）不得被落盘，因为该分区即将写入新数据，落盘会导致写入状态不一致。原文还称“引擎会静默忽略对此分区的 SPILL 动作”，但保存的 executeActions 未实现该拦截；调度器必须自行排除该目标。保护只针对本次 overflow 动作列表；INSERT 完成后的 periodic 可以落盘刚写入的分区。
同一分区可被多次落盘和加载，每次 SPILL 或 LOAD 均按上述公式独立计算 I/O 耗时。例如某分区经历了 SPILL→LOAD→SPILL→LOAD，则产生 2 次写 I/O + 2 次读 I/O 的总开销。
跨算子落盘：调度器在处理任意算子的溢出或 miss 时，可以选择落盘任何算子（包括其他处于 BUILD 或 PROBE 阶段的算子）的 in_memory 分区来释放内存（但须遵守上一条对当前目标分区的限制）。
参考引擎已实现的无效动作处理：以下动作被静默忽略，不产生任何 I/O 开销，也不会释放内存：
对 in_memory = false 或 data_size = 0 的分区执行 SPILL
对 in_memory = true 的分区执行 LOAD
对 BUILD 阶段算子的分区执行 LOAD
原文将“overflow 时 SPILL 当前 INSERT 目标”也列为静默忽略，但代码没有实现，不能列入上述已实现行为。

调度器应返回符合文字约束的动作，并自行保证 ID 有效、overflow 只含 SPILL 且排除目标、miss 末项为 LOAD 目标。参考代码并不完整校验这些条件；无效 ID 甚至可能越界访问，并非安全忽略。

【解释补充：动作执行】动作按列表顺序逐项执行，每次 LOAD 均必须满足容量；不支持原子回滚。空分区 SPILL 被忽略且仍保持驻留。LOAD 后再 SPILL 仍按完整 data_size 写盘，没有“磁盘保留副本所以免费驱逐”。磁盘分区 INSERT 则只支付新增字节的写盘成本。

【解释补充：释放量与目标保护】overflow 所需额外释放量为 max(0, memory_used + tuple_size - M)，已有空闲空间应计入。文字禁止 SPILL 当前 INSERT 目标，但参考 executeActions 缺少相应拦截；策略主动排除该目标，不依赖漏检查。miss 最后一项仍按明确要求 LOAD 当前目标。

2.5 调度器接口

参赛者需实现一个调度器类，核心接口如下：

【原代码注释勘误】下列接口代码保留原文。“引擎会静默忽略目标 SPILL”的注释与 executeActions 不符；禁止该动作仍是必须遵守的文字规则，由调度器主动满足。miss 末项 LOAD 目标也是文字要求，引擎只检查动作执行后目标驻留，并未检查末项或动作类型集合。

```
// 调度动作
struct Action {
    enum Type { SPILL, LOAD } type;
    int op_id;          // 目标算子ID
    int partition_id;   // 目标分区ID
};
class SpillScheduler {
public:
    virtual ~SpillScheduler() = default;
    // 场景1：INSERT操作导致内存溢出时调用
    // 参数：当前系统状态、触发操作的算子ID、目标分区ID、待插入元组大小
    // 返回：需要执行的SPILL动作列表（至少释放足够空间）
    // 注意：禁止落盘当前正在插入的目标分区（即 op_id + partition_id 对应的分区），
    //       因为该分区即将写入新数据，落盘会导致写入丢失。引擎会静默忽略对此分区的 SPILL 动作。
    virtual std::vector<Action> onMemoryOverflow(
        const SystemState& state,
        int op_id,
        int partition_id,
        int64_t tuple_size) = 0;
    // 场景2：PROBE操作需要加载磁盘分区时调用
    // 参数：当前系统状态、触发操作的算子ID、目标分区ID
    // 返回：需要执行的SPILL+LOAD动作列表（最后一条必须为LOAD目标分区）
    virtual std::vector<Action> onPartitionMiss(
        const SystemState& state,
        int op_id,
        int partition_id) = 0;
    // 场景3：周期性调度，每处理B个操作后调用
    // 返回：任意SPILL/LOAD动作列表（允许主动预取或提前落盘）
    virtual std::vector<Action> onPeriodicAdjust(
        const SystemState& state) = 0;
};
```

SystemState 为只读视图，提供以下信息：
memory_used, memory_limit
各算子的 phase 和所有分区的 data_size / in_memory 状态
当前已执行的 hit_count, miss_count, 累计 I/O 字节数
next_op：下一个待处理操作的类型（INSERT / SWITCH / PROBE）及参数（用于预取决策），若当前已是最后一个操作则为空
在线调度约束：
本赛题要求严格在线调度。调度器在每次回调时，只能依赖 SystemState 中提供的信息（含 next_op 中的下一个待处理操作）做出决策。
禁止通过任何方式访问完整操作序列（如框架内部的 op_lines、输入文件流等），禁止预先读取全部操作后进行离线规划。
选手可在 MySpillScheduler 内部维护历史统计信息（如各分区累计被 Probe 的次数、历史命中率等），这些信息须由调度器在历次回调中自行累积，不得来源于对未来的预读。
next_op 仅提供紧邻的下一条操作，调度器可据此做一步预取/提前落盘决策，但不得假设能获取更远的未来操作。

【解释补充：可见信息】next_op 是当前输入行的下一条输入，不是下一次回调。当前输入的多个回调看到相同 next_op；最后一条输入时 type 为空。普通 hit/INSERT 没有逐条通知，不能把反复展示的 next_op 累加为真实访问次数；相同内容也可能属于不同输入。真实 miss_penalty、N、B、build_count/probe_count 不在 SystemState 中，不自行注入。历史只能记合法可见的观察。

【解释补充：Naive 平局】参考 Naive 用 std::sort 按大小降序，未指定等大候选的稳定顺序。基线保留原代码及编译环境；自有策略明确自己的确定性平局规则。

2.6 样例说明

参数：K=1, P=3, M=256 字节, W=100 字节/μs, R=80 字节/μs, miss_penalty=10μs
Build 阶段（6 个元组，每个 64 字节）：

步骤	操作	操作完成后的驻留分区	memory_used	说明
1	INSERT key=0	p0=64	64	key % 3 = 0
2	INSERT key=3	p0=128	128	key % 3 = 0
3	INSERT key=1	p0=128, p1=64	192	
4	INSERT key=4	p0=128, p1=128	256	内存满
5	SPILL p0；INSERT key=2	p1=128, p2=64	192	需释放 64；实际释放 128，写盘 1.28μs
6	INSERT key=5	p1=128, p2=128	256	p0 在磁盘

SWITCH op0，进入 PROBE 阶段。原示例的一种朴素策略轨迹如下（等大分区的平局选择依该示例）：

步骤	Probe key	目标	结果	动作	I/O 时间
7	0	p0	Miss	SPILL p2；LOAD p0	2.88μs
8	1	p1	Hit	无	0
9	2	p2	Miss	SPILL p0；LOAD p2	2.88μs
10	3	p0	Miss	SPILL p2；LOAD p0	2.88μs

统计：
Hit = 1，Miss = 3
总 I/O 耗时 = 1.28(build spill) + 2.88×3(probe) = 9.92μs
总惩罚 = I/O耗时 + miss_penalty × miss_count = 9.92 + 10×3 = 39.92μs
原文错误对比（保留作勘误依据）：“若步骤5落盘 p1 而非 p0……仅 PROBE key=2 需加载 p2，Hit=2、Miss=1”。这不是本版采用的结论，正确轨迹如下。

> 【解释补充：纠正上段对比】上段原文 Hit=2、Miss=1 与四次 Probe 不守恒，且落盘 p1 后 key=1 必须 Miss，p2 最初仍驻留。一个合法改进轨迹是：key=0 Hit；key=1 时 SPILL p2、LOAD p1；key=2 时 SPILL p1、LOAD p2；key=3 Hit。结果 Hit=2、Miss=2，I/O=1.28+2×2.88=7.04μs，I/O+Miss 惩罚=27.04μs，另加实际决策时间。这不是唯一合法策略结果。

三、期望达到的指标和效果（含性能/精度等指标）

功能目标：
在给定内存约束下正确执行全部工作负载，不出现 OOM。
正确统计命中/未命中次数和磁盘 I/O 开销。
所有分区状态转换逻辑正确，memory_used 在任意时刻不超过 M。
性能目标：
数据规模：K ≤ 8 个并发算子，P ≤ 64 个分区，总操作数 ≤ 5,000,000。
内存上限 M：64MB ~ 4GB。
评估公式：Score = Total_IO_Time + miss_penalty × miss_count + Total_Decision_Time，越小越优。
Total_IO_Time：所有 SPILL 全量写、LOAD 全量读及磁盘态 INSERT 增量写的 I/O 耗时之和（微秒）。
total_io_bytes 同样累计三者的字节数。
miss_count：所有 PROBE 操作中分区未命中的总次数。
miss_penalty：每次未命中的惩罚值（由测试数据给出）。
Total_Decision_Time：调度器在所有 onMemoryOverflow / onPartitionMiss / onPeriodicAdjust 调用中累计测得的经过时间之和（原文称“CPU 时间”，但参考代码并非专用 CPU 计时；微秒，单位与 I/O 时间一致，便于直接相加）。该项将决策开销纳入验收指标，避免选手采用过重的搜索/迭代算法以换取微小的 miss 减少。
硬件环境：8 核 CPU、8GB 内存、256GB SSD。所有选手程序在相同的评测机上运行，确保 Total_Decision_Time 的横向可比性。
时间限制：单个测试用例的墙上时间（wall-clock）限制为 60 秒。原文称“包含 I/O 模拟时间与决策 CPU 时间”；参考实现的模拟 I/O 仅累计数值，并不实际等待。真实输入解析、状态处理和决策等均消耗进程时间。选手调度器的单次决策（onMemoryOverflow / onPartitionMiss / onPeriodicAdjust）复杂度建议不超过 O(K·P·log(K·P))。若墙上时间超限，该测试用例得 0 分。
评分说明：评分同时考虑 I/O 耗时、miss 指标和调度器决策 CPU 开销三者之和。选手需在"减少 miss 与 I/O"和"控制决策复杂度"之间取得平衡：过于简单的策略会导致 miss 过多，过于复杂的策略会因 Total_Decision_Time 过高而拉高成本。原文称“仅计回调 CPU 耗时、框架自身不计入”，实际计时边界如下。

> 【解释补充：计时与墙时】采用参考 timedSchedulerCall 原边界：测量 high_resolution_clock 的经过时间，不是专用进程 CPU 时间；getSystemState 的构造、operators 深拷贝及临时状态销毁也在计时范围内，动作执行、输入解析不在 Total_Decision_Time 中。模拟 I/O 数值只是成本累计，不实际 sleep，不把它直接加到真实墙时上判超时。60 秒属于外部评测约束，参考 main 没有超时控制；具体启动/终止计时边界未由此代码定义。
>
> 【解释补充：硬件已给定部分】原题已写 8 核 CPU、8GB 内存、256GB SSD。不能将这些说成完全未知；精确 CPU 型号等未给条件见文末。

四、测试用例

用例获取与验证：下面提供提示性用例（含样例输入及小规模场景用例），供选手本地熟悉输入输出格式、验证功能正确性。选手亦可依据下述用例格式自行构造用例进行自测。比赛结束提交代码后，由评测系统在统一环境上运行全量评测用例（含未公开用例）并计算最终成绩。

输入格式

第一行包含 6 个参数：
K P M W R miss_penalty
K：算子数量（1 ≤ K ≤ 8）
P：每算子分区数（2 ≤ P ≤ 64）
M：内存上限（字节，64MB ≤ M ≤ 4GB）
W：磁盘顺序写带宽（字节/微秒，1 ≤ W ≤ 10000）
R：磁盘顺序读带宽（字节/μs，1 ≤ R ≤ 10000）
miss_penalty：每次 Probe 未命中的惩罚值（微秒，1 ≤ miss_penalty ≤ 1000）
第二行包含两个整数：
N B
N：操作序列总条数（1 ≤ N ≤ 5000000）
B：周期性调度的间隔（每 B 个操作触发一次 onPeriodicAdjust，1 ≤ B ≤ 10000）
接下来 K 行，每行 3 个整数：
op_id build_count probe_count
op_id：算子编号（0 ≤ op_id < K）
build_count：该算子的 Build 元组数
probe_count：该算子的 Probe 元组数

> 【解释补充：输入兼容前提】原题未明确要求这 K 行的顺序；参考 addOperator 按输入顺序 push_back，其他位置却按 op_id 索引，未建立 ID 映射。为兼容保存代码，本地构造输入按 op_id=0..K-1 升序且唯一；这不是已确认的官方新增输入保证。build_count/probe_count 在 main 读取后未使用，本地数据仍保持与实际操作一致。容量、缺口与累计字节使用 int64_t。

接下来 N 行，每行描述一个操作：
INSERT op_id key size
SWITCH op_id
PROBE op_id key
INSERT：插入元组，0 ≤ key ≤ 10^9，1 ≤ size ≤ 1048576
SWITCH：切换算子阶段
PROBE：探测键
输入约束：
每个 operator 的 INSERT 操作必须在其 SWITCH 之前。
每个 operator 的 PROBE 操作必须在其 SWITCH 之后。
每条 key 对应的 partition_id = key % P，输入保证数据倾斜存在（部分分区远大于其他分区）。
保证任意分区的最终 data_size ≤ M（即任意单个分区可以被完整加载到内存中），不会出现单分区超过内存上限导致无解的情况。
保证每个测试用例至少存在一种合法调度方案，使全部操作在 memory_used ≤ M 约束下正确完成（即不存在结构性无解的用例）。
不同算子的操作可以任意交替。例如算子 0 已进入 PROBE 阶段时，算子 1 可能仍在 BUILD 阶段；调度器需处理混合阶段的场景。

输出格式

程序输出到标准输出，格式如下：

```
Total_IO_Time: <value>
Total_Decision_Time: <value>
Hit_Count: <value>
Miss_Count: <value>
Total_Score: <value>
```

Total_IO_Time：SPILL 全量写、LOAD 全量读和磁盘态 INSERT 增量写的 I/O 耗时之和（微秒，保留 2 位小数）。
Total_Decision_Time：三种回调计时区间的累计经过时间（微秒，保留 2 位小数），由 high_resolution_clock 测量，包含状态复制，具体边界见第三节。
Hit_Count：Probe 命中总次数。
Miss_Count：Probe 未命中总次数。

Total_Score = Total_IO_Time + miss_penalty × Miss_Count + Total_Decision_Time。

【解释补充：输出舍入】公式作用于未舍入的内部 double，之后各项分别保留两位小数。因此显示分项之和可能与显示 Total_Score 相差 0.01，不能要求文本数值精确相等。

样例输入

【转换修复】原 PDF 的样例在第 5–6 页之间连续，跨页不是输入空行。本版移除提取产生的三行空白；参考 main 按 N 读取物理行，不跳过空行，以下操作区应恰好为 11 行。M=256 小于正式范围，且三个最终分区均为128字节、不满足文字所述大小倾斜；本版将其解释为格式与轨迹演示，不用于推断正式测试范围。

```
1 3 256 100 80 10
11 3
0 6 4
INSERT 0 0 64
INSERT 0 3 64
INSERT 0 1 64
INSERT 0 4 64
INSERT 0 2 64
INSERT 0 5 64
SWITCH 0
PROBE 0 0
PROBE 0 1
PROBE 0 2
PROBE 0 3
```

样例输出

```
Total_IO_Time: 9.92
Total_Decision_Time: 0.50
Hit_Count: 1
Miss_Count: 3
Total_Score: 40.42
```

说明：样例中 Total_Decision_Time: 0.50 为示意值，实际值取决于评测机的 CPU 性能与调度器实现。该值由框架代码自动测量并输出，调度器无需自行打印。Total_Score = 9.92 + 10 × 3 + 0.50 = 40.42。

【解释补充：样例输出成立的条件】按2.6节指定的平局选择，Build 写出128字节，Probe再写出3×128字节并读入3×128字节，所以 I/O=512/100+384/80=9.92μs，Hit=1、Miss=3。B=3 时，10次 INSERT/PROBE 在第3、6、9次后触发3次 periodic，另有1次 overflow 和3次 miss，共7次回调；SWITCH 不计数。0.50μs 是这7次计时之和的示意值，真实运行不保证复现，Total_Score 应为39.92+实测决策时间。std::sort 未规定等大平局，因此文中轨迹不是每个编译环境必然走出的唯一轨迹；其他合法策略也可得到不同 Hit/Miss 与 I/O。样例非计时断言。

参考框架代码

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>
#include <cstdint>
#include <cassert>
#include <sstream>
#include <iomanip>
#include <chrono>
#include <memory>
#include <tuple>
/* ===== 数据结构定义 ===== */
enum class Phase { BUILD, PROBE };
struct PartitionState {
    int partition_id;
    int64_t data_size;   // 当前数据总大小（字节）
    bool in_memory;      // 是否在内存中
    PartitionState() : partition_id(-1), data_size(0), in_memory(true) {}
};
struct OperatorState {
    int op_id;
    Phase phase;
    std::vector<PartitionState> partitions;  // P个分区
};
struct PendingOp {
    std::string type;      // "INSERT" / "SWITCH" / "PROBE" / ""（无后续操作）
```

```
    int op_id;             // 算子ID，type为空时无意义
    int64_t key;           // INSERT/PROBE时的key，SWITCH时无意义
    int64_t tuple_size;    // INSERT时的元组大小，其余操作无意义
    PendingOp() : op_id(-1), key(0), tuple_size(0) {}
};
struct SystemState {
    int64_t memory_used;
    int64_t memory_limit;
    int num_partitions;
    double write_bandwidth;    // 字节/微秒
    double read_bandwidth;     // 字节/微秒
    int64_t hit_count;
    int64_t miss_count;
    int64_t total_io_bytes;
    std::vector<OperatorState> operators;
    PendingOp next_op;         // 下一个待处理操作

};
/* ===== 调度动作 ===== */
struct Action {
    enum Type { SPILL, LOAD } type;
    int op_id;
    int partition_id;
    Action(Type t, int o, int p) : type(t), op_id(o), partition_id(p) {}
};
/* ===== 调度器抽象基类 ===== */
class SpillScheduler {
public:
    virtual ~SpillScheduler() = default;
    // 场景1：INSERT导致内存溢出时调用
    virtual std::vector<Action> onMemoryOverflow(
        const SystemState& state,
        int op_id, int partition_id, int64_t tuple_size) = 0;
    // 场景2：PROBE需要加载磁盘分区时调用
    virtual std::vector<Action> onPartitionMiss(
        const SystemState& state,
        int op_id, int partition_id) = 0;
    // 场景3：周期性调度，每处理B个操作后调用
    virtual std::vector<Action> onPeriodicAdjust(
        const SystemState& state) = 0;
};
/* ===== 朴素调度器（基线实现）===== */
class NaiveSpillScheduler : public SpillScheduler {
public:
    // 内存溢出时：选择最大的in_memory分区落盘，直到释放足够空间
    std::vector<Action> onMemoryOverflow(
        const SystemState& state,
        int op_id, int partition_id, int64_t tuple_size) override {
        std::vector<Action> actions;
        int64_t needed = tuple_size;
        int64_t available = state.memory_limit - state.memory_used;
        // 收集所有in_memory分区，按data_size降序排列（排除当前正在插入的目标分区）
        std::vector<std::tuple<int64_t, int, int>> candidates;
        for (const auto& op : state.operators) {
            for (const auto& part : op.partitions) {
                if (part.in_memory && part.data_size > 0 &&
                    !(op.op_id == op_id && part.partition_id == partition_id)) {
                    candidates.emplace_back(part.data_size, op.op_id, part.partition_id);
                }
            }
        }
        std::sort(candidates.begin(), candidates.end(),
                  [](const auto& a, const auto& b) { return std::get<0>(a) > std::get<0>(b); });
        for (const auto& cand : candidates) {
            if (available >= needed) break;
            int64_t sz = std::get<0>(cand);

            int oid = std::get<1>(cand);
            int pid = std::get<2>(cand);
            actions.emplace_back(Action::SPILL, oid, pid);
            available += sz;
        }
        return actions;
    }
    // Probe未命中时：选择最大的in_memory分区落盘腾出空间，然后加载目标分区
    std::vector<Action> onPartitionMiss(
        const SystemState& state,
        int op_id, int partition_id) override {
        std::vector<Action> actions;
        int64_t needed = state.operators[op_id].partitions[partition_id].data_size;
        int64_t available = state.memory_limit - state.memory_used;
        if (available < needed) {
            std::vector<std::tuple<int64_t, int, int>> candidates;
            for (const auto& op : state.operators) {
```

```
                for (const auto& part : op.partitions) {
                    if (part.in_memory && part.data_size > 0 &&
                        !(op.op_id == op_id && part.partition_id == partition_id)) {
                        candidates.emplace_back(part.data_size, op.op_id, part.partition_id);
                    }
                }
            }
            std::sort(candidates.begin(), candidates.end(),
                      [](const auto& a, const auto& b) { return std::get<0>(a) > std::get<0>(b); });
            for (const auto& cand : candidates) {
                if (available >= needed) break;
                int64_t sz = std::get<0>(cand);
                int oid = std::get<1>(cand);
                int pid = std::get<2>(cand);
                actions.emplace_back(Action::SPILL, oid, pid);
                available += sz;
            }
        }
        actions.emplace_back(Action::LOAD, op_id, partition_id);
        return actions;
    }
    // 朴素策略：不做主动调整
    std::vector<Action> onPeriodicAdjust(const SystemState& state) override {
        return {};
    }
};
/* ===== 执行引擎 ===== */
class HashJoinEngine {
private:
    int64_t memory_limit_;
    int num_partitions_;
    double write_bandwidth_;
    double read_bandwidth_;
    int miss_penalty_;
    int periodic_interval_;

    int64_t memory_used_;
    int64_t hit_count_;
    int64_t miss_count_;
    double total_io_time_;
    double total_decision_time_;   // 调度器决策CPU耗时累计（微秒）
    int64_t total_io_bytes_;
    int operation_count_;
    std::vector<OperatorState> operators_;
    std::unique_ptr<SpillScheduler> scheduler_;
    PendingOp next_op_;  // 预存下一个操作，供getSystemState使用
    SystemState getSystemState() const {
        SystemState state;
        state.memory_used = memory_used_;
        state.memory_limit = memory_limit_;
        state.num_partitions = num_partitions_;
        state.write_bandwidth = write_bandwidth_;
        state.read_bandwidth = read_bandwidth_;
        state.hit_count = hit_count_;
        state.miss_count = miss_count_;
        state.total_io_bytes = total_io_bytes_;
        state.operators = operators_;
        state.next_op = next_op_;
        return state;
    }
    // 测量并累加一次调度器调用的CPU耗时（微秒）
    template<typename F>
    auto timedSchedulerCall(F&& f) -> decltype(f()) {
        auto t_start = std::chrono::high_resolution_clock::now();
        auto result = f();
        auto t_end = std::chrono::high_resolution_clock::now();
        double us = std::chrono::duration<double, std::micro>(t_end - t_start).count();
        total_decision_time_ += us;
        return result;
    }
    void executeActions(const std::vector<Action>& actions) {
        for (const auto& act : actions) {
            auto& part = operators_[act.op_id].partitions[act.partition_id];
            if (act.type == Action::SPILL) {
                if (!part.in_memory || part.data_size == 0) continue;
                total_io_time_ += static_cast<double>(part.data_size) / write_bandwidth_;
                total_io_bytes_ += part.data_size;
                part.in_memory = false;
                memory_used_ -= part.data_size;
            } else if (act.type == Action::LOAD) {
                // 引擎静默忽略非法 LOAD：对 in_memory 分区、或 BUILD 阶段算子的分区
                if (part.in_memory) continue;
                if (operators_[act.op_id].phase == Phase::BUILD) continue;
                if (memory_used_ + part.data_size > memory_limit_) {
```

```
                    std::cerr << "OOM: LOAD would exceed memory limit" << std::endl;
                    std::exit(1);
                }
                total_io_time_ += static_cast<double>(part.data_size) / read_bandwidth_;
                total_io_bytes_ += part.data_size;
                part.in_memory = true;
                memory_used_ += part.data_size;
            }
        }

    }
public:
    HashJoinEngine(int64_t memory_limit, int num_partitions,
                   double write_bandwidth, double read_bandwidth,
                   int miss_penalty, int periodic_interval,
                   std::unique_ptr<SpillScheduler> scheduler)
        : memory_limit_(memory_limit), num_partitions_(num_partitions),
          write_bandwidth_(write_bandwidth), read_bandwidth_(read_bandwidth),
          miss_penalty_(miss_penalty), periodic_interval_(periodic_interval),
          memory_used_(0), hit_count_(0), miss_count_(0),
          total_io_time_(0.0), total_decision_time_(0.0),
          total_io_bytes_(0), operation_count_(0),
          scheduler_(std::move(scheduler)) {}
    void addOperator(int op_id) {
        OperatorState op;
        op.op_id = op_id;
        op.phase = Phase::BUILD;
        op.partitions.resize(num_partitions_);
        for (int i = 0; i < num_partitions_; ++i) {
            op.partitions[i].partition_id = i;
        }
        operators_.push_back(op);
    }
    // 设置下一个待处理操作（由main循环在执行当前操作前调用）
    void setNextOp(const PendingOp& op) { next_op_ = op; }
    void processInsert(int op_id, int64_t key, int64_t tuple_size) {
        int pid = key % num_partitions_;
        auto& part = operators_[op_id].partitions[pid];
        if (part.in_memory) {
            if (memory_used_ + tuple_size > memory_limit_) {
                // 内存溢出，调用调度器（计时）
                auto actions = timedSchedulerCall([&]() {
                    return scheduler_->onMemoryOverflow(
                        getSystemState(), op_id, pid, tuple_size);
                });
                executeActions(actions);
                // 校验：调度器释放的空间是否足够
                if (memory_used_ + tuple_size > memory_limit_) {
                    std::cerr << "OOM: scheduler failed to free enough memory for INSERT" << std::endl;
                    std::exit(1);
                }
            }
            part.data_size += tuple_size;
            memory_used_ += tuple_size;
        } else {
            // 分区在磁盘上，元组直接写磁盘
            total_io_time_ += static_cast<double>(tuple_size) / write_bandwidth_;
            total_io_bytes_ += tuple_size;
            part.data_size += tuple_size;
        }
        operation_count_++;
        if (operation_count_ % periodic_interval_ == 0) {
            auto actions = timedSchedulerCall([&]() {
                return scheduler_->onPeriodicAdjust(getSystemState());
            });

            executeActions(actions);
        }
    }
    // SWITCH不计入operation_count_，不触发周期性调度
    void processSwitch(int op_id) {
        operators_[op_id].phase = Phase::PROBE;
    }
    void processProbe(int op_id, int64_t key) {
        int pid = key % num_partitions_;
        auto& part = operators_[op_id].partitions[pid];
        if (part.in_memory) {
            hit_count_++;
        } else {
            miss_count_++;
            // 调用调度器加载分区（计时）
            auto actions = timedSchedulerCall([&]() {
                return scheduler_->onPartitionMiss(
                    getSystemState(), op_id, pid);
```

```
            });
            executeActions(actions);
            // 校验：调度器必须将目标分区加载到内存
            if (!part.in_memory) {
                std::cerr << "OOM: scheduler failed to load target partition for PROBE" << std::endl;
                std::exit(1);
            }
        }
        operation_count_++;
        if (operation_count_ % periodic_interval_ == 0) {
            auto actions = timedSchedulerCall([&]() {
                return scheduler_->onPeriodicAdjust(getSystemState());
            });
            executeActions(actions);
        }
    }
    void printResult() const {
        double total_score = total_io_time_ +
            static_cast<double>(miss_penalty_) * miss_count_ + total_decision_time_;
        std::cout << std::fixed << std::setprecision(2);
        std::cout << "Total_IO_Time: " << total_io_time_ << std::endl;
        std::cout << "Total_Decision_Time: " << total_decision_time_ << std::endl;
        std::cout << "Hit_Count: " << hit_count_ << std::endl;
        std::cout << "Miss_Count: " << miss_count_ << std::endl;
        std::cout << "Total_Score: " << total_score << std::endl;
    }
};
/* ===== 主函数 ===== */
int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: " << argv[0] << " <input_file>" << std::endl;
        return 1;
    }
    std::ifstream fin(argv[1]);
    if (!fin) {

        std::cerr << "Cannot open input file: " << argv[1] << std::endl;
        return 1;
    }
    int K, P, miss_penalty, N, B;
    int64_t M;
    double W, R;
    fin >> K >> P >> M >> W >> R >> miss_penalty;
    fin >> N >> B;
    auto scheduler = std::make_unique<NaiveSpillScheduler>();
    HashJoinEngine engine(M, P, W, R, miss_penalty, B, std::move(scheduler));
    for (int i = 0; i < K; ++i) {
        int op_id, build_count, probe_count;
        fin >> op_id >> build_count >> probe_count;
        engine.addOperator(op_id);
    }
    // 预读所有操作行
    std::vector<std::string> op_lines;
    {
        std::string line;
        std::getline(fin, line);  // consume newline
        for (int i = 0; i < N; ++i) {
            std::getline(fin, line);
            op_lines.push_back(line);
        }
    }
    for (int i = 0; i < N; ++i) {
        // 设置next_op为当前操作的下一条
        PendingOp next;
        if (i + 1 < N) {
            std::istringstream niss(op_lines[i + 1]);
            niss >> next.type;
            if (next.type == "INSERT") niss >> next.op_id >> next.key >> next.tuple_size;
            else if (next.type == "SWITCH") niss >> next.op_id;
            else if (next.type == "PROBE") niss >> next.op_id >> next.key;
        } else {
            next.type = "";  // 无后续操作
        }
        engine.setNextOp(next);
        // 执行当前操作
        std::istringstream iss(op_lines[i]);
        std::string op;
        iss >> op;
        if (op == "INSERT") {
            int op_id;
            int64_t key, size;
            iss >> op_id >> key >> size;
            engine.processInsert(op_id, key, size);
        } else if (op == "SWITCH") {
```

```
            int op_id;
            iss >> op_id;
            engine.processSwitch(op_id);
        } else if (op == "PROBE") {
            int op_id;

            int64_t key;
            iss >> op_id >> key;
            engine.processProbe(op_id, key);
        }
    }
    engine.printResult();
    return 0;
}
```

参赛者实现指引

只需实现一个类：MySpillScheduler，继承自 SpillScheduler，实现以下三个纯虚接口。其余代码（执行引擎、I/O 模拟、状态管理、输入解析、决策时间测量、结果输出）均由框架提供，选手不得修改。

接口	触发时机	返回要求
onMemoryOverflow	目标驻留且 INSERT 将超出 M	SPILL 列表，释放足够空间，排除当前目标
onPartitionMiss	PROBE 目标不驻留	必要时先腾空间，最后 LOAD 当前目标；可只含一次 LOAD
onPeriodicAdjust	每 B 个 INSERT/PROBE 后	任意合法 SPILL/LOAD，可为空

类名约定：参赛者实现的子类必须命名为 MySpillScheduler。评测系统会以选手提交的源文件替换框架中的调度器实现，并在 main 函数中通过 std::make_unique<MySpillScheduler>() 实例化。选手提交的代码中不应包含 main 函数，也不应修改框架代码中除调度器外的任何部分。
提交与编译约束：
提交范围：选手仅提交 MySpillScheduler 的实现（可含辅助类/函数，但须在同一提交文件中）。框架代码（含 HashJoinEngine、main、所有数据结构定义）由评测系统提供，选手不得修改。
禁止修改框架：不得修改 HashJoinEngine、main 函数、SystemState/Action/PartitionState 等数据结构定义，不得在普通 INSERT/PROBE 命中时新增逐次通知（仍须保留计数达到 B 时已有的周期回调）。
禁止第三方库：仅允许使用 C++ 标准库，禁止引入任何第三方库（如 Google OR-Tools、Eigen、Boost 等）或外部求解器。
编译命令：由评测系统统一编译，命令固定为 g++ -std=c++14 -O2，选手无需自行指定编译选项。
提交格式：单个 .cpp 文件，文件名不限，内含 MySpillScheduler 类定义及成员函数实现。

> 【原代码注释勘误】下面骨架的“须释放至少 tuple_size”不精确：只需释放 max(0, memory_used + tuple_size - M)。例如 M=100、已用90、待插20时只缺10。保留代码注释以便逐字核对原版；实现使用缺口公式，且不得 SPILL 当前 INSERT 目标。

考生代码骨架（仅此部分由选手实现，接口中无任何实现）：

```
/* ===== 选手提交：MySpillScheduler ===== */
class MySpillScheduler : public SpillScheduler {
public:
    // 可按需添加构造函数、成员变量、辅助函数
    // MySpillScheduler() {}
    // 场景1：INSERT 导致内存溢出时调用
    // 参数：当前系统只读状态、触发算子ID、目标分区ID、待插入元组大小
    // 返回：SPILL 动作列表（须释放至少 tuple_size 字节的空间）
    std::vector<Action> onMemoryOverflow(
        const SystemState& state,
        int op_id,
        int partition_id,
        int64_t tuple_size) override;
    // 场景2：PROBE 需要加载磁盘分区时调用
    // 参数：当前系统只读状态、触发算子ID、目标分区ID
    // 返回：SPILL+LOAD 动作列表（最后一条必须为 LOAD 目标分区）
    std::vector<Action> onPartitionMiss(
        const SystemState& state,
        int op_id,
        int partition_id) override;
    // 场景3：每处理 B 个操作后周期性调用

    // 返回：任意 SPILL/LOAD 动作列表（允许为空）
    std::vector<Action> onPeriodicAdjust(
```

```
        const SystemState& state) override;
};
// 实现上述三个函数的函数体
std::vector<Action> MySpillScheduler::onMemoryOverflow(
    const SystemState& state,
    int op_id,
    int partition_id,
    int64_t tuple_size) {
    // TODO: 选手实现
    return {};
}
std::vector<Action> MySpillScheduler::onPartitionMiss(
    const SystemState& state,
    int op_id,
    int partition_id) {
    // TODO: 选手实现
    return {};
}
std::vector<Action> MySpillScheduler::onPeriodicAdjust(
    const SystemState& state) {
    // TODO: 选手实现
    return {};
}
```

main 函数中的替换点（仅供说明，选手无需修改此处，评测系统会自动链接）：

```
// 原始：auto scheduler = std::make_unique<NaiveSpillScheduler>();
// 替换为：auto scheduler = std::make_unique<MySpillScheduler>();
```

说明：
框架代码已实现完整的执行引擎和朴素调度策略（NaiveSpillScheduler）。
朴素策略的行为：内存溢出时落盘最大分区；Probe 未命中时落盘最大分区腾空间后加载目标分区；不做主动调整。
参赛者需实现 MySpillScheduler 替换 NaiveSpillScheduler，优化落盘/加载决策。框架中的 NaiveSpillScheduler 仅作为基线参考，选手可保留以便本地对比，但提交时评测系统只编译 MySpillScheduler。
引擎在执行调度器返回的动作后，会校验内存约束和分区状态。若 INSERT 后仍超出内存上限、LOAD 后超出内存上限、或 PROBE 后目标分区仍不在内存中，引擎将输出 OOM 错误并退出（比赛评测时判定为 OOM，该用例得 0 分）。
SWITCH 操作不计入周期性调度的操作计数（即只有 INSERT 和 PROBE 计入 B 的计数）。
决策时间测量：timedSchedulerCall 用 high_resolution_clock 测量三种回调的经过时间，累加为 total_decision_time_（微秒），输出为 Total_Decision_Time 并计入 Total_Score。实际包含 getSystemState 状态构造、深拷贝与临时状态销毁；输入解析、动作执行等在计时区间外。原文将其称为“仅调度器 CPU 耗时、框架自身不计入”，与该边界不完全一致。所有选手程序按原题要求在相同评测机上运行，实际比较还需绑定编译和运行条件。
测试用例：评测用例由评测系统在比赛时下发，选手无需也无法提前获取实际测试数据。赛题文档中的"样例输入"仅供格式演示之用，不含任何评测数据。

五、参考文献（可选，如有请提供）

1. Spilling-Based Parallelism for Hash Joins — Proceedings of the VLDB Endowment, 2023

2. Cache-Oblivious Hash Joins — CIDR, 2022

3. Grace Hash Join — DeWitt & Katz, 1983（经典多趟 Hash Join 算法）

4. DuckDB Query Execution Engine — DuckDB Documentation, Vectorized Execution & Spilling

六、评选标准（如评分维度，分成几档等）

基础要求（Level 0，50 分）：
所有功能模块实现逻辑正确，严格遵循内存约束（总内存不超限、分区的落盘/加载状态转换正确），命中/未命中统计精准无误，输出结果格式完整合规。代码需基于 C++ 实现，不出现崩溃（core dump）、挂死（hang）等问题。在朴素策略基础上，需至少实现一种有别于朴素策略的优化手段。
效能要求（Level 1，50 分）：

以组委会提供的基线程序（NaiveSpillScheduler）在各测试数据集上的 Total_Score 为 BASE，选手的 Total_Score 为 VAL。每组测试数据得分按以下公式计算，单组得分范围为 0~10 分：
single_score_i = clamp(10 × (1 - VAL_i / BASE_i), 0, 10)
VAL 越小（越优），得分越高；VAL = BASE（与基线持平）时得 0 分。
若 VAL_i = 0（选手得分异常为 0），该组数据得 0 分。
若 VAL_i ≥ BASE_i（未优于基线），该组数据得 0 分。
若 BASE_i = 0（基线本身无 I/O 且无 miss），则该组数据：VAL_i = 0 时记 0 分，VAL_i > 0 时记 0 分（即基线为 0 的用例不参与 Level 1 计分）。
若出现 OOM/超时，该组数据得 0 分。
Level 1 总分为所有测试数据得分之和，上限 50 分：
Level1_Score = min(50, Σ single_score_i)
自适应要求（Level 2，加分项，最多 +30 分）：
具备动态感知数据倾斜与 Probe 访问模式的能力，可根据实时状态自适应调整落盘与预取策略，在数据显著倾斜、多算子并发竞争内存、Probe 局部性变化等场景下仍能维持稳定性能。需在提交代码中附简要说明文档阐述策略思路，包括：所感知的指标、感知方式、调整决策的触发条件与规则。由专家评审打分，单项最高加 30 分。
创新要求（Level 3，加分项，最多 +30 分）：
提出具有理论或实践创新性的解决方案。以下方向仅为提示，考生可自行选择任何创新方向：
基于代价估计的分区落盘策略；
基于 next_op 与历史访问模式的预取策略；
跨算子内存配额的动态调度；
其他具有理论或实践创新性的调度算法。
创新点需通过与朴素基线（NaiveSpillScheduler）的对比实验验证其有效性（在组委会提供的测试用例上展示 Score 提升，并分析提升来源），并附算法说明文档与复杂度分析（单次决策复杂度建议不超过 O(K·P·log(K·P))）。由专家评审打分，单项最高加 30 分。
总分上限：Level 0 + Level 1 + Level 2 + Level 3 = 50 + 50 + 30 + 30 = 160 分。

七、文字要求与参考代码的差异及剩余边界

下表区分“文字要求”与“保存代码实际实现”。选手仍须主动遵守文字约束；代码漏检查不是许可。本版保留原框架，不通过改动引擎补齐校验，亦不声称隐藏评测一定沿用这些遗漏。

文字要求或描述	参考代码实际行为	本版采用的处理
overflow 禁止 SPILL 当前 INSERT 目标，引擎会忽略	executeActions 不接收当前目标，未拦截；processInsert 已进入驻留分支，目标被落盘后仍增加 memory_used，可能破坏状态一致性	策略主动排除目标；不能依赖静默忽略
overflow 只返回 SPILL；miss 动作最后为 LOAD 当前目标	不按回调检查动作类型或末项；miss 只检查最终目标驻留	主动满足回调约束；空间足够时可只返回 LOAD
动作引用有效算子与分区	executeActions 直接用下标访问，未检查 ID 范围	自行保证合法 ID；越界不是可忽略动作
INSERT 在 SWITCH 前，PROBE 在 SWITCH 后	processInsert/processProbe 不检查 phase；processSwitch 仅赋值，无重复切换检查；BUILD LOAD 则确有检查	输入按单次 BUILD→PROBE 构造，不利用宽松解析；零访问、切换次数等未完整形式化的范围另记
头部计数和操作数描述工作负载，输入满足数值范围	build_count/probe_count 读后不用；getline 读取 N 个物理行，不检查每行解析成功、额外行或数值范围	保持计数一致，操作区不放空行，不能把未报错当输入合法证明
op_id 标识算子	addOperator 按行 push_back，未按 ID 建映射或排序	本地按0..K-1排列；官方输入行序未明确
决策时间为回调 CPU 时间，框架耗时不计入	用时钟差计经过时间，状态构造、深拷贝与临时状态销毁也计入	使用实际计时边界，分清模拟成本与真实墙时
单例60秒、OOM/超时计0分、统一编译和最终分级评分	参考 main 没有超时控制、沙箱、统一编译或 Level 0/1/2/3 评分；只输出原始成本，部分失败 exit(1)	这些是外部评测要求，不能用 main 运行成功证明满足
在线信息只来自回调及 next_op，不得预读未来	main 为执行需要预读全部操作，但没有隔离机制阻止参赛代码绕过接口	main 的预读不等于调度器可访问；只使用合法可见信息

原骨架“至少释放 tuple_size”是注释不精确，参考 Naive 与容量检查实际使用已有空闲量，详见前文。原样例的错误对比和矛盾的计时描述均可在原 PDF 直接看到，并非本次转换造成。

剩余边界：

1. 未完整形式化的输入范围：build_count/probe_count 无明确下界，SWITCH 是否恰好一次、PROBE key 的单独取值范围没有完整给出；参考解析器也不补足这些保证。本地单次 BUILD→PROBE、非负 key 等作为兼容约定，不冒称官方新增限制。

2. 隐藏分布与倾斜：正式文字要求有大小倾斜，但没有量化阈值或热点稳定性保证；均衡数据可以作诊断，不宣称满足该定性要求。不能由均衡小样例推断隐藏输入分布。

3. 字节范围：MB/GB 换算未明确；执行直接使用输入字节整数，容量与累计字节用64位。

4. 评测细节：已给8核CPU、8GB内存、256GB SSD与60秒；具体CPU/编译器版本、进程资源配额、超时起止边界、重复及聚合方法未完整给定。Naive源码及构建应绑定，不把平局差异当算法增益。

5. 评分边界：Total_Score为成本，越低越好；Level 1分数越高越好。人工Level 0/2/3无法由成本自动推出，团队汇总不等于官方新增规定。“BASE=0即无I/O且无miss”还应同时考虑决策时间，三项非负时总成本为0需三项都为0。

八、转换、换页与核验说明

• 原 PDF 为15页，指纹与本版来源一致。逐页核查原始正文提取的保留/排除行：未发现因主栏裁剪丢失题干正文；被排除内容为网页导航、互动栏、页眉页脚、更新时间等。表格的线性抽取与自动结构化副本有重复和错列，本版按原 PDF 可见表格重新排版。

• 样例跨原第5–6页。本版已移除转换引入的空行；不是修改原始工作负载。正文补充插入中断原句的错误也已修复。INSERT步骤重复编号是原PDF已有问题，本版顺序整理为1、2、3。

• 三个C++块与冻结Markdown逐块去空白一致，并对照原PDF逐页代码行（包括跨页续行）检查。原代码注释中的规范差异在块外显著注明；不修改框架逻辑，不补入引擎检查。

• 修复 Markdown 将裸露的 <value>、<MySpillScheduler>、<NaiveSpillScheduler> 误识别为 HTML、旧渲染器随之漏掉它们的问题：输出格式与替换点使用代码围栏，正文模板表达式使用行内代码；渲染器也保留意外出现的原始 HTML 字面文本。旧渲染器遗漏的列表编号与项目符号现已补齐。

• 本版PDF由同一Markdown重新排版，页数可与原PDF不同，换页不代表操作或代码边界。渲染保留列表编号、代码空格，表格跨页重复表头；短样例代码块尽量整块保留，长代码可跨页。

• 样例小规模执行用于验证解析和输出含义，决策时间仅记录观察、不作性能结论。具体验证和产物指纹见同目录 STATEMENT_ENHANCED.manifest.json 与 STATEMENT_REVIEW.md。

• 原文参考文献保留但未独立核验，本次语义检查不依赖其真实性；未重新核验官网或隐藏评测实现。